ПРАКТИЧЕСКОЕ ЗАДАНИЕ / ОТЧЁТ

Мониторинг API и инфраструктура

Bash · анализ HTTP-трафика · Git Workflow · Docker Compose · GitHub Actions

Версия для проверки. Реализация и CI готовы. До окончательной сдачи требуется добавить скриншот терминала с **docker-compose ps**. Его реальный текстовый вывод сохранён на стр. 5. Управление приложениями остановлено пользователем до получения снимка.

Этап	Проверенный результат
Git и GitHub	Публичный репозиторий, защищённая main, feature/monitoring, открытый PR #1
Bash	Два исполняемых скрипта, 5 локальных тестов; генерация 360 строк
Docker Compose	Два работающих сервиса; proxy healthy; ответ Monitoring Service Active
GitHub Actions	Два успешных запуска; синтаксис, тесты, публичный API и Compose
Сетевой анализ	Реальный дамп: 42 пакета, 8 HTTP-пакетов; GET/POST и 200/404/503

Репозиторий: https://github.com/chavonadelal/infra-monitoring-project
Pull Request: PR #1 - реализация и проверка
Дамп: скачать http-traffic.pcapng. Этот же файл вложен в PDF как attachment.

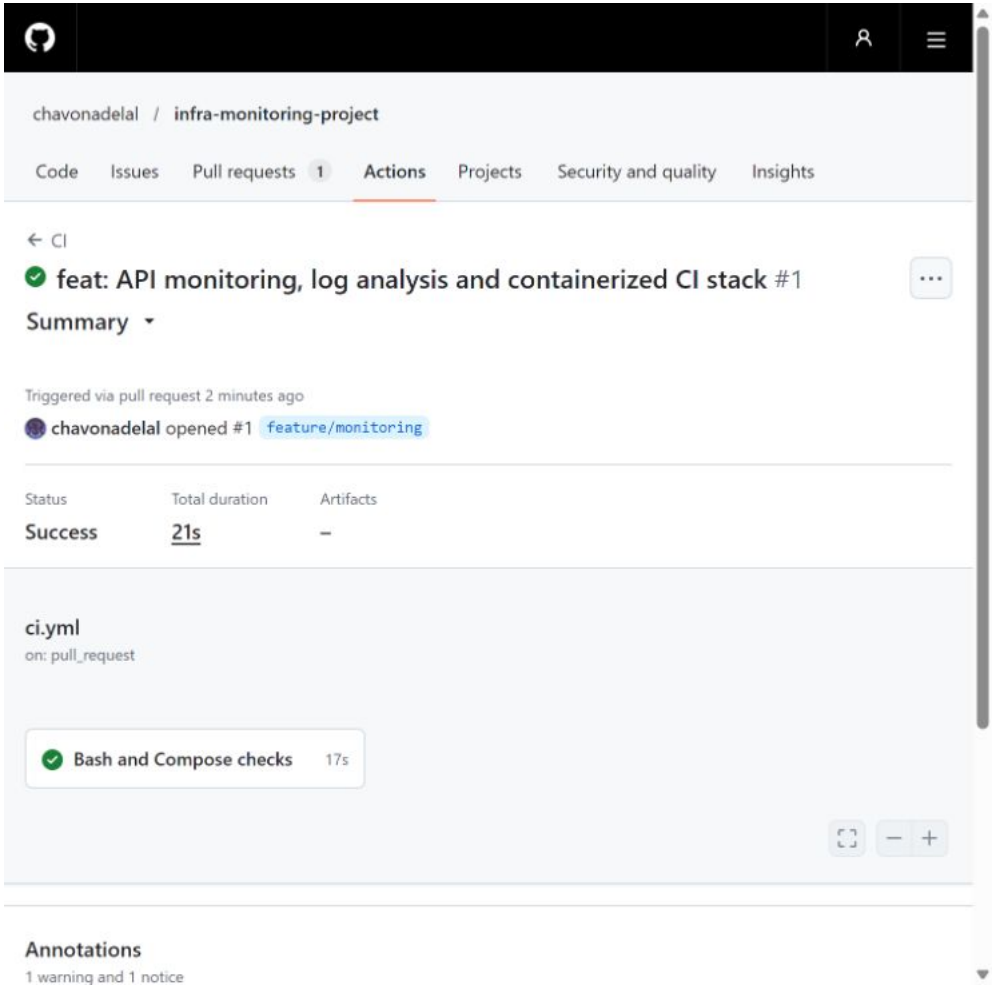
Окружение, Git и успешный CI

Рабочая станция: Windows, Git Bash 5.2.37 и Git 2.53.0. Docker Desktop 4.86.0 использует Linux Engine 29.7.2. Wireshark 4.6.9 извлечён из официального подписанного MSI и запущен на Windows.

Стартовый коммит находится в **main**. Скрипты, Compose, CI и документация добавлены осмысленными коммитами в **feature/monitoring**. PR оставлен открытым.

Защита main требует Pull Request, успешный статус **Bash and Compose checks** и разрешённые обсуждения. Защита действует и для администратора; force push и удаление ветки запрещены.

Последний проверенный CI: run 43427063415, commit bfe2c58, результат **success**. Справа - скриншот первого успешного запуска (43425871215, commit bc4176d). Последующий запуск также проверяет демонстрационные шаги.



Анализ access.log: 360 запросов

Фактический вывод analyze_logs.sh в Git Bash. Это текстовый протокол выполнения, сохранённый без изменения чисел; исходник доступен в docs/evidence/logs-output.txt.

```
infra-monitoring-project | branch: feature/monitoring

$ ./analyze_logs.sh --generate
Создан тестовый файл: access.log (360 строк)
Общее количество запросов: 360
Топ-3 IP-адреса (запросы IP):
120 192.168.1.10
90 10.0.0.5
60 172.16.0.3
Запросов с кодами 4xx: 72
Запросов с кодами 5xx: 72
exit code: 0

$ wc -l access.log
360 access.log

$ head -n 2 access.log
192.168.1.10 - - [15/Aug/2026:10:00:00] "GET /api/users HTTP/1.1" 200
192.168.1.10 - - [15/Aug/2026:10:00:01] "GET /api/users HTTP/1.1" 201
```

Пайплайн: sed нормализует CRLF → grep проверяет формат и IPv4 regexp → awk выделяет поля → sort | uniq -c | sort строит топ-3. Второй awk считает 4xx/5xx. Некорректные записи исключаются; например, IP 999.1.1.1 тест не принимает.

Проверка доступности API

Реальные вызовы curl через check_api.sh: доступный локальный и публичный URL, ошибочный HTTP-статус и невалидный URL. Показаны фактические коды завершения.

```
infra-monitoring-project | branch: feature/monitoring

$ ./check_api.sh http://localhost:80
API доступен
exit code: 0

$ ./check_api.sh https://httpbin.org/status/200
API доступен
exit code: 0

$ ./check_api.sh https://httpbin.org/status/404
API недоступен, код ответа: 404
exit code: 1

$ ./check_api.sh not-a-url
Использование: ./check_api.sh http[s]://адрес/api
API недоступен, код ответа: 000 (некорректный URL)
exit code: 1
```

Только HTTP 200 приводит к exit code 0. При HTTP 404, неверном аргументе или сетевом сбое скрипт возвращает 1. Таймауты: 5 секунд на подключение и 15 секунд на запрос. Сетевой сбой выводится как 000. Redirect не считается успешным ответом.

Контейнеры и проверка Nginx

Маршрут запроса: localhost:80 → proxy (nginx:alpine) → api:8080 (hashicorp/http-echo:1.0.0). Сервисы объединены отдельной bridge-сетью monitoring. Конфигурация Nginx подключена read-only; наружу опубликован только порт 80.

Фактический docker-compose ps (компактный табличный формат; полный вывод сохранён в docs/evidence/docker-output.txt):

```
$ docker-compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
SERVICE   IMAGE                        STATUS                    PORTS
api       hashicorp/http-echo:1.0.0    Up 12 minutes             5678/tcp, 8080/tcp
proxy     nginx:alpine                 Up 12 minutes (healthy)   0.0.0.0:80->80/tcp, [::]:80->80/tcp
```

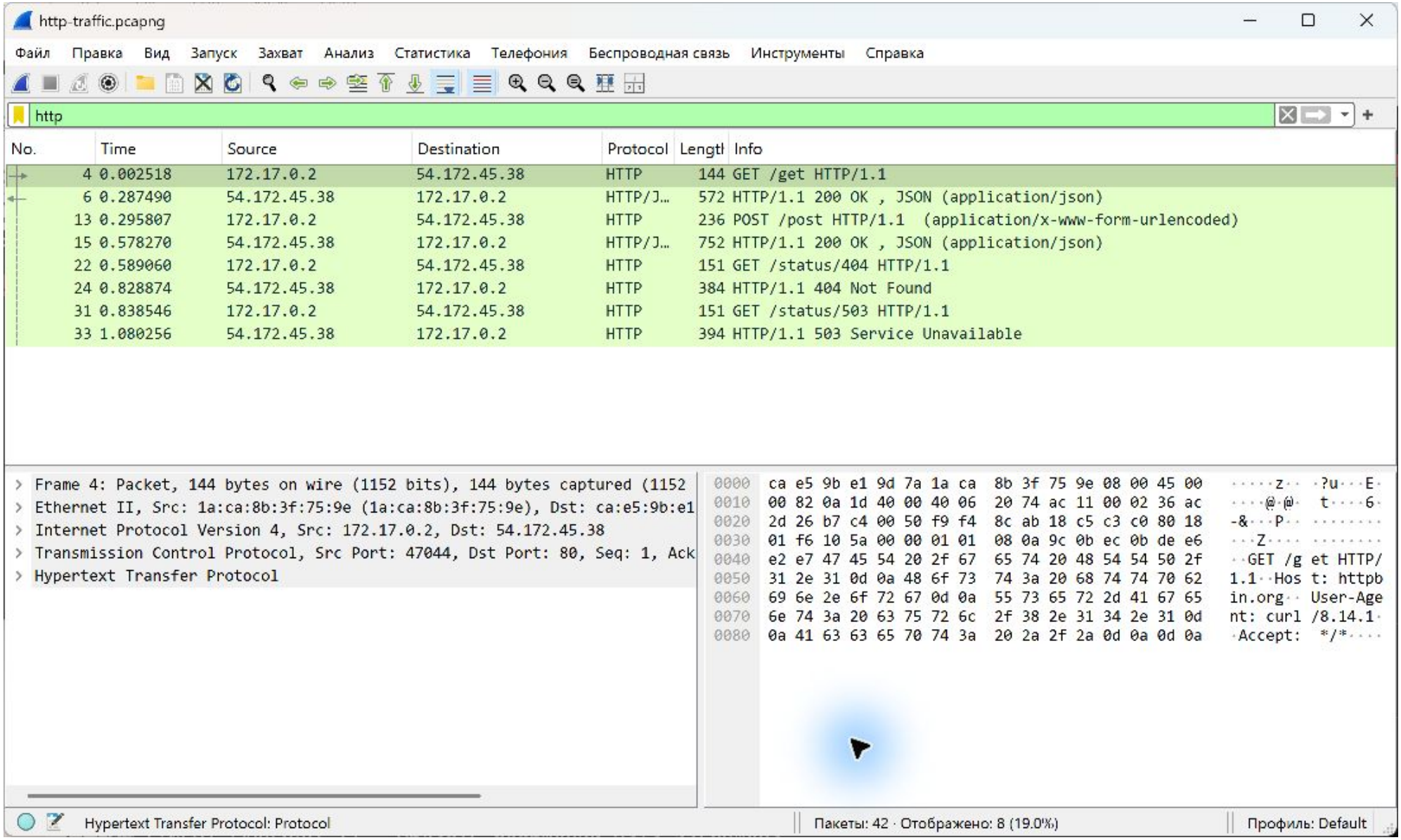
```
$ curl --fail --silent --show-error http://localhost:80
Monitoring Service Active

$ docker compose exec proxy nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Healthcheck прокси сравнивает ответ с ожидаемой строкой, проверяя всю цепочку Nginx → API. Аналогичный запуск и проверка ответа успешно выполнены в CI.

Требуется перед сдачей: добавить настоящий скриншот терминала с docker-compose ps. Текстовый вывод выше подтверждает результат, но не заменяет требуемый снимок экрана.

Wireshark: HTTP-трафик



Настоящий скриншот Wireshark. Зелёный фильтр http; отображено 8 пакетов из 42. В таблице видны IP, GET/POST и ответы HTTP 200/404/503.

Разбор дампа и способ захвата

Захват выполнен tcpdump на сетевом интерфейсе eth0 временного Linux-контейнера Docker Desktop. Это реальный исходящий трафик через Docker NAT. На Windows драйвер Npcap не установлен; Wireshark использован для просмотра и фильтрации дампа. Пакеты не генерировались искусственно.

Участник	Адрес и роль
HTTP-клиент	172.17.0.2 - адрес eth0 тестового контейнера, порты клиента динамические
Публичный API	54.172.45.38:80 - httpbin.org на момент захвата

Метод	Путь	Ответ	Пакеты запроса / ответа
GET	/get	200 OK	4 / 6
POST	/post	200 OK	13 / 15
GET	/status/404	404 Not Found	22 / 24
GET	/status/503	503 Service Unavailable	31 / 33

Фильтр захвата: **tcp port 80**. Формат: pcapng, 6616 байт, 42 Ethernet-пакета. В HTTP видны методы и коды; в HTTPS эти поля зашифрованы. Исходный pcap преобразован editcap -F pcapng без изменения пакетов.
SHA-256: 154906d7045b108bd9030fbd65bd4efa1258c911437c98dbcc2daaabde3b7aca

SSH/SCP, воспроизведение и файлы сдачи

README содержит запуск, остановку, тестирование и примеры безопасного деплоя. Удалённый сервер в задании не задан: команды ниже приведены как инструкция; реальный SSH-деплой не выполнялся.

```
ssh-keygen -t ed25519 -a 100 -C "infra-monitoring-deploy" -f ~/.ssh/infra_monitoring
ssh-copy-id -i ~/.ssh/infra_monitoring.pub deploy@example.org
git archive --format=tar.gz --output=../infra-monitoring-project.tar.gz HEAD
scp -i ~/.ssh/infra_monitoring ../infra-monitoring-project.tar.gz deploy@example.org:~/
ssh -i ~/.ssh/infra_monitoring deploy@example.org
```

Используется приватный ключ с парольной фразой. Публичный ключ устанавливается на сервер; fingerprint проверяется по независимому каналу. Проверка ключа хоста не отключается. В репозиторий не входят логи, приватные ключи, .env и артефакты сборки.

Файл / ссылка	Назначение
feature/monitoring	Исходники и документация; main защищена
Pull Request #1	Описание изменений и результаты проверок
GitHub Actions	Успешный CI: Bash, тесты, httpbin и Compose
http-traffic.pcapng	Дамп доступен по ссылке и вложен в этот PDF

Источники: официальный дистрибутив Wireshark; HashiCorp http-echo. Результаты команд и снимки экрана получены при выполнении этого проекта.